

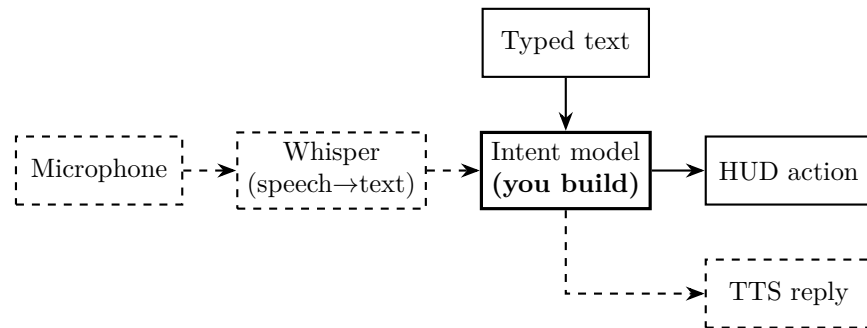
CLAWS Onboarding: EVA Voice Intent Model

CLAWS Voice Assistant Team

Fall 2026

Goal: write training sentences for 36 EVA intents, fine-tune a small model, classify sentences it has never seen. Text in, intent out is required. Voice (Whisper + TTS) is a stretch goal. You write all the code yourself; `code/` has the data validator plus short outlines of each step. Your work goes in your own folder.

1 Where this fits



Solid = required. Dashed = stretch. The model picks an intent (plus a name for five intents), or `unhandled` if the speech isn't a command. The HUD reads the actual value from telemetry.

Reviewed and merged by: Ayush.

2 The intents

Labels are **case-sensitive**. Copy them exactly. Also in `code/intents.json` and a copy in your folder. `unhandled` is a real intent: write 30+ things an astronaut might say that aren't commands ("nice view", "copy that", "hold on").

Intent	Meaning	Name
<code>get_oxy_pri_storage</code>	Primary oxygen storage %	
<code>get_oxy_sec_storage</code>	Secondary oxygen storage %	
<code>get_oxy_pri_pressure</code>	Primary oxygen tank pressure	
<code>get_oxy_sec_pressure</code>	Secondary oxygen tank pressure	
<code>get_suit_pressure_oxy</code>	Suit O ₂ partial pressure	
<code>get_suit_pressure_co2</code>	Suit CO ₂ partial pressure	
<code>get_suit_pressure_other</code>	Suit trace gas pressure	
<code>get_suit_pressure_total</code>	Total suit pressure	
<code>get_helmet_pressure_co2</code>	Helmet CO ₂ pressure	
<code>get_fan_pri_rpm</code>	Primary fan speed (RPM)	
<code>get_fan_sec_rpm</code>	Secondary fan speed (RPM)	
<code>get_scrubber_a_co2_storage</code>	Scrubber A CO ₂ storage	
<code>get_scrubber_b_co2_storage</code>	Scrubber B CO ₂ storage	
<code>get_temperature</code>	Suit internal temperature	
<code>get_coolant_storage</code>	Coolant storage %	
<code>get_coolant_gas_pressure</code>	Coolant gas loop pressure	
<code>get_coolant_liquid_pressure</code>	Coolant liquid loop pressure	
<code>get_heart_rate</code>	Astronaut heart rate	
<code>get_oxy_consumption</code>	Oxygen consumption rate	
<code>get_co2_production</code>	CO ₂ production rate	
<code>get_eva_elapsed_time</code>	Elapsed mission time	
<code>get_primary_battery_level</code>	Primary battery %	
<code>get_secondary_battery_level</code>	Secondary battery %	
<code>get_warnings</code>	Active warnings and alerts	
<code>open_menu_vitals</code>	Open vitals panel	
<code>open_menu_navigation</code>	Open navigation / map panel	
<code>open_menu_tasks</code>	Open tasks / checklist panel	
<code>open_menu_uia</code>	Open UIA airlock panel	
<code>close_menu</code>	Close the active panel	
<code>Get_coordinates</code>	Coordinates of a named target	✓
<code>Set_navigation_target</code>	Navigate to a named place	✓
<code>reroute_navigation</code>	Compute a new route	
<code>Add_waypoint</code>	Add a named waypoint	✓
<code>Delete_waypoint</code>	Delete a named waypoint	✓
<code>Complete_task</code>	Mark a named task complete	✓
<code>unhandled</code>	Not a command: small talk, noise, off-topic	

3 Setup

Any laptop works (CPU is fine; MPS/CUDA used if present). Use **Python 3.10–3.12** (newer versions may lack PyTorch builds). Your folder is already made:

Member	Folder
Aaron	submissions/aaron/
Evan	submissions/evan/
Gloria	submissions/gloria/
Raiana	submissions/raiana/
Sam	submissions/sam/
Utsav	submissions/utsav/
Vivian	submissions/vivian/

```
git clone https://github.com/CLAWS-UMICH/AI-Onboarding-2026.git
cd AI-Onboarding-2026
git checkout -b onboarding/<your-name>
```

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cd submissions/<your-name>           # run everything from here
```

Windows (PowerShell)

```
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
cd submissions\<your-name>
```

If PowerShell refuses to run `Activate.ps1`, run this once:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

In Command Prompt use `.venv\Scripts\activate.bat`. After activating, `python` works the same on every OS and all commands below use forward slashes, which Windows accepts.

Each folder has an empty `data/train.jsonl`, an empty `runs/`, a `notes.md` template, `MODELS.md` (the model options below) and a copy of `intents.json`. Write your scripts in your folder and run them from there. Structure them however you like (one file or many, notebook, any library in `requirements.txt` or beyond). The outlines in `code/` say what each step should do, not how. Never edit `code/`. Your PR should only touch your own folder.

4 Write the training data

The model is only as good as your sentences. One file, `data/train.jsonl`, one JSON object per line, exactly one intent each:

```
{ "text": "what's my primary oxygen at", "intents": ["get_oxy_pri_storage"] }
{ "text": "O2 secondary tank pressure?", "intents": ["get_oxy_sec_pressure"] }
{ "text": "uh how hot is it in the suit", "intents": ["get_temperature"] }
{ "text": "pull up the map", "intents": ["open_menu_navigation"] }
{ "text": "drop a waypoint called Crater Rim", "intents": ["Add_waypoint"] }
{ "text": "copy that, heading out", "intents": ["unhandled"] }
{ "text": "whats the heart rate", "intents": ["get_heart_rate"], "source": "llm" }
```

Rules

- **30+ per intent** (50 ideal). Write them by hand, generate them synthetically (e.g. with an LLM), or mix both. Check every generated row and tag it `"source": "llm"`. The validator counts both.
- Save the file as UTF-8 (VS Code does by default).
- Mix questions, commands and clipped phrases (“fan RPM?”). 2 to 20 words.
- Vary words: oxygen/O2/ox, battery/charge/power, menu/panel/screen.
- Sound like speech: “uh”, “please”, lowercase, no punctuation.
- Always say which one: primary/secondary, A/B, gas/liquid.
- Vary names in the five name intents. No near-duplicates.

Listing 1: `validate.py`

```
"""Check a training file: valid JSON, known labels, one label per row, no duplicates,
    enough rows."""
import json
import sys
from collections import Counter
from pathlib import Path

MIN_PER_INTENT = 30    # rows per intent; handwritten or synthetic both count

if len(sys.argv) != 2:
    sys.exit("usage: python validate.py data/train.jsonl")

intents_file = Path(__file__).with_name("intents.json")
labels = {x["intent"] for x in json.loads(intents_file.read_text(encoding="utf-8"))}
seen, total, problems = set(), Counter(), []

for n, line in enumerate(open(sys.argv[1], encoding="utf-8"), 1):
    if not line.strip():
        continue
    try:
        row = json.loads(line)
        text, intents = row["text"].strip(), row["intents"]
    except (ValueError, KeyError, TypeError, AttributeError):
```

```

        problems.append(f'line {n}: not a valid row, expected {{"text": "...", "intents":
["label"]}}}')
        continue
    if not isinstance(intents, list) or len(intents) != 1 or intents[0] not in labels:
        problems.append(f"line {n}: bad intents {intents!r} (need exactly one label from
intents.json)")
        continue
    if text.lower() in seen:
        problems.append(f"line {n}: duplicate text '{text}'")
    seen.add(text.lower())
    total[intents[0]] += 1

for label in sorted(labels):
    if total[label] < MIN_PER_INTENT:
        problems.append(f"{label}: only {total[label]} rows (need {MIN_PER_INTENT})")

print("\n".join(problems))
print(f"{sum(total.values())} rows, {len(problems)} problems")
sys.exit(1 if problems else 0)

```

```
python ../../code/validate.py data/train.jsonl    # fix until "0 problems"
```

5 Build the model

Pick an encoder. Three options, all on Hugging Face. MiniLM is the default; try another once your data is solid. Click a name for its model page (huggingface.co/<model>).

Model	Params	Hidden	Notes
sentence-transformers/all-MiniLM-L6-v2	22M	384	default, fastest
BAAI/bge-small-en-v1.5	33M	384	often more accurate
sentence-transformers/all-mpnet-base-v2	110M	768	biggest, $\sim 5\times$ slower

Use the same encoder for your baseline and your fine-tune so you compare like with like. Changed models? Re-run both.

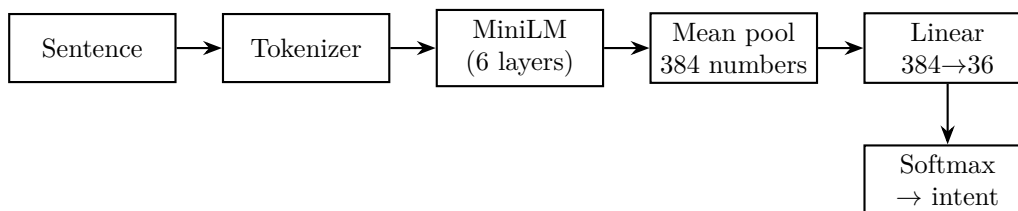
Step 0: baseline. Frozen sentence embeddings + logistic regression. This is the number to beat.

Listing 2: code/baseline.py (outline)

```
"""Step 0: baseline. No fine-tuning. Write your own version in your folder.

Outline (do it however you like):
- Load data/train.jsonl, hold out ~15% (stratified, fixed seed).
- Embed sentences with a frozen encoder (sentence-transformers is installed).
- Fit a simple classifier (e.g. sklearn logistic regression) on the embeddings.
- Print a per-intent report and save it to baseline.txt (include the model name).
"""
```

Step 1: fine-tune. One shape that works: encoder + mean pooling + one linear layer (sizes below are for MiniLM). Try something else if you want.



Listing 3: code/model.py (outline)

```
"""The intent model. Write your own version in your folder.

Outline (do it however you like):
- A pretrained encoder from Hugging Face (MiniLM, BGE-small, MPNet; see MODELS.md).
- Pool the token outputs into one vector per sentence (mean pooling works well).
- A classification head that maps that vector to one score per intent.
- Use MPS/CUDA if available, otherwise CPU.
"""
```

Hold out the same split as your baseline. Print accuracy as you train and list every miss at the end. Expect >0.95 and a win over the baseline; if not, fix your data.

Listing 4: code/train.py (outline)

```
"""Step 1: fine-tune. Write your own version in your folder.

Outline (do it however you like):
- Load data/train.jsonl and hold out the same ~15% split as your baseline.
- Map labels to ids; train the encoder + head with cross-entropy.
- Report accuracy and macro-F1 on the held-out set; print every miss.
- Save to runs/: model weights (model.pt, not committed), label2id.json,
  and metrics.json with at least model, accuracy, macro_f1.
"""
```

6 Evaluate and predict

Listing 5: code/predict.py (outline)

```
"""Classify a sentence. Write your own version in your folder.

Outline (do it however you like):
- Load your saved model and label map from runs/.
- Take a sentence from the command line, print JSON with the text,
  selected_intent and confidence.
- Low confidence -> answer "unhandled" instead of guessing.
- For the five name intents (Get_coordinates, Set_navigation_target, Add_waypoint,
  Delete_waypoint, Complete_task) also pull out the name as "slot".
"""
```

Output can look like this:

```
python predict.py "add a waypoint named Olympus Ridge"
{"text": "add a waypoint named Olympus Ridge", "selected_intent": "Add_waypoint",
 "confidence": 0.969, "slot": "Olympus Ridge"}
```

Pick a confidence threshold below which you answer **unhandled**.

Held-out accuracy is not the real test, because you wrote both halves. The real test: write 10+ new sentences you didn't plan for and see what breaks. Common confusions: primary vs. secondary, scrubber A vs. B, coolant gas vs. liquid, opening the nav menu vs. rerouting, and the coordinate/target/waypoint intents. Fix confusions with better data, not code.

Names: for the five name intents, pull the name out of the sentence ("add a waypoint named Olympus Ridge" → **Olympus Ridge**). Watch for cases like "mark task soil sample complete". Any approach is fine.

7 Stretch: make it talk

```
pip install faster-whisper sounddevice pyttsx3    # one option
```

macOS asks for microphone permission for your terminal the first time. Allow it, then restart the terminal. macOS has a built-in `say` command; `pyttsx3` works on Windows and Linux.

Listing 6: `code/voice.py` (outline)

```
"""Stretch goal: microphone -> speech-to-text -> your intent model -> spoken reply.
Write your own version in your folder.

Outline (do it however you like):
- Record a few seconds of audio (e.g. sounddevice).
- Transcribe it (e.g. faster-whisper; a prompt with domain words like
  "EVA, UIA, C02, scrubber, waypoint" helps spelling).
- Run your predict function on the text.
- Speak a reply (macOS `say`, or pyttsx3 on Windows/Linux).
"""
```

Next steps: `small.en` Whisper, speak the real value back, push-to-talk.

8 What to hand in

One big PR. Commit and push as often as you like, but open a single PR only when everything below is done. No partial PRs. Only touch your own folder, so the seven PRs never conflict.

- ☐ Your code (any structure)
- ☐ `data/train.jsonl`: 0 problems in `validate.py`
- ☐ `runs/metrics.json`: fine-tuned model beats `baseline.txt`
- ☐ `runs/metrics.json` has at least `model`, `accuracy`, `macro_f1`
- ☐ `runs/label2id.json`, `baseline.txt`
- ☐ `notes.md`: model used, results, 10+ unseen sentences, what failed
- ☐ **Not** `runs/model.pt` (`.gitignore` skips it)

The PR is done when every box is checked. Ayush reviews and merges. Stuck for over an hour? Ask in the channel.

Editing this guide: change the `.tex`, rebuild, and commit both the `.tex` and the PDF.

```
latexmk -pdf claws_intent_onboarding.tex
```